

SUKKUR IBA UNIVERSITY

Department of Computer Science

Web Engineering (CSE-311)

Lab Report 05: JavaScript Fundamentals

Pure Node.js, Variables, Functions, Arrays, Objects & Tip Calculator

Student Name:	Muhammad Tayyab
CMS ID:	023-24-0118
Course:	Web Engineering (CSE-311)
Semester:	5th Semester (Fall 2026)
Submitted To:	Lab Instructor
Repository:	github.com/taybro-o/web-engineering-lab

1. Warm-up (warmup.js)

Declared const for name, let for score, double(n) function, and logged the 2nd array item.

File: Week 5/warmup.js

```
const name = "Muhammad Tayyab";
let score = 0;

function double(n) {
  return n * 2;
}

const skills = ["JavaScript", "CSS", "HTML"];

console.log("Name:", name, "| Score:", score);
console.log("Double 21:", double(21));
console.log("Second skill:", skills[1]);
```

2. Class Activities A, B & C (basics.js)

- **Activity A:** Checked const reassignment error, incremented count with let, and checked equality (0 == false is true, 0 === false is false, [] == false is true).
- **Activity B:** Implemented getInitials, isPalindrome, and countWords.
- **Activity C:** Implemented letterGrade(score) and displayCount(n) using ?? so 0 is kept.

File: Week 5/basics.js

```
// Class Activity A
const course = "Web Eng";
try {
  course = "Other";
} catch (e) {
  console.log("Cannot reassign const:", e.message);
}

let count = 0;
count++;
count++;
console.log("Count:", count);

// 0 == false is true because loose equality converts false to 0
console.log(0 == false);
// 0 === false is false because types differ (number vs boolean)
console.log(0 === false);
// [] == false is true because [] converts to "" then to 0
console.log([] == false);

// Class Activity B
const getInitials = (name) =>
  name.trim().split(/\s+/).map((w) => w[0].toUpperCase()).join("");

const isPalindrome = (str) => {
  const s = str.toLowerCase().replace(/^[a-z0-9]/g, "");
  return s.length > 0 && s === s.split("").reverse().join("");
};

const countWords = (str) =>
  str.trim() === "" ? 0 : str.trim().split(/\s+/).length;

console.log("Initials:", getInitials("Ayesha Khan"));
console.log("Palindrome:", isPalindrome("Race car"));
console.log("Word count:", countWords("the quick brown fox"));
```

```
// Class Activity C
const letterGrade = (score) => {
  if (score >= 90) return "A";
  if (score >= 80) return "B";
  if (score >= 70) return "C";
  if (score >= 60) return "D";
  return "F";
};

const displayCount = (n) => n ?? "N/A";

console.log("Grade 85:", letterGrade(85));
console.log("displayCount(0):", displayCount(0));
console.log("displayCount(null):", displayCount(null));
console.log("displayCount(42):", displayCount(42));
```

3. Class Activity D: Students Pipeline (arrays.js)

Used reduce for average, filter and map for above average, some and every checks, and sorted a copy without mutating the original.

File: Week 5/arrays.js

```
// Class Activity D
const students = [
  { name: "Ali", score: 82 },
  { name: "Sara", score: 95 },
  { name: "Ahmed", score: 68 },
  { name: "Zainab", score: 77 },
  { name: "Bilal", score: 91 },
  { name: "Hamza", score: 45 }
];

// 1. Above average names
const total = students.reduce((sum, s) => sum + s.score, 0);
const avg = total / students.length;
const aboveAvg = students.filter((s) => s.score > avg).map((s) => s.name);

console.log("Class Average:", avg.toFixed(2));
console.log("Above average students:", aboveAvg);

// 2. some & every
console.log("Any score > 90:", students.some((s) => s.score > 90));
console.log("All scores > 40:", students.every((s) => s.score > 40));

// 3. Sort high to low without mutating original
const sorted = [...students].sort((a, b) => b.score - a.score);
console.log("Sorted:", sorted);
console.log("Original untouched:", students[0].name === "Ali");
```

4. Class Activity E: Products & groupBy (objects.js)

Filtered inStock products, reduced total inventory price, and grouped items by category using reduce.

File: Week 5/objects.js

```
// Class Activity E
const products = [
  { id: 1, title: "Mechanical Keyboard", price: 85, category: "peripherals", inStock: true },
  { id: 2, title: "USB-C Cable", price: 15, category: "cables", inStock: true },
  { id: 3, title: "Gaming Mouse", price: 55, category: "peripherals", inStock: false },
  { id: 4, title: "HDMI Cable", price: 25, category: "cables", inStock: true },
  { id: 5, title: "Headphones", price: 120, category: "audio", inStock: true }
];

// 1. inStock titles
const inStock = products.filter((p) => p.inStock).map((p) => p.title);
console.log("In stock:", inStock);

// 2. Total price
const total = products.reduce((sum, p) => sum + p.price, 0);
console.log("Total price:", total);

// 3. groupBy using reduce
const groupBy = (arr, key) =>
  arr.reduce((acc, item) => {
    const k = item[key];
    if (!acc[k]) acc[k] = [];
    acc[k].push(item);
    return acc;
  }, {});

console.log("Grouped by category:", groupBy(products, "category"));
```

5. Class Activity F & Homework Task 4: Tip Calculator (calculator.js)

Wrote calculateBill with parameter validation, wrapped error cases in try/catch, and added formatBill receipt formatter.

File: Week 5/calculator.js

```
// Class Activity F & Homework Task 4
function calculateBill({ amount, tipPercent = 15, people = 1 } = {}) {
  if (typeof amount !== "number" || amount <= 0) {
    throw new Error("Amount must be a number greater than 0");
  }
  if (typeof tipPercent !== "number" || tipPercent < 0) {
    throw new Error("Tip percentage must be a valid non-negative number");
  }
  if (!Number.isInteger(people) || people < 1) {
    throw new Error("People must be an integer of 1 or more");
  }

  const tipAmount = amount * (tipPercent / 100);
  const totalAmount = amount + tipAmount;

  return {
    amount: Number(amount.toFixed(2)),
    tipPercent,
```

```

        tipAmount: Number(tipAmount.toFixed(2)),
        totalAmount: Number(totalAmount.toFixed(2)),
        people,
        perPerson: Number((totalAmount / people).toFixed(2))
    };
}

function formatBill(b) {
    return `Subtotal: PKR ${b.amount}\nTip (${b.tipPercent}%): PKR ${b.tipAmount}\nTotal: PKR ${b.totalAmount}\n
    People: ${b.people}\nPer person: PKR ${b.perPerson}`;
}

// Happy path
const bill = calculateBill({ amount: 2400, tipPercent: 10, people: 3 });
console.log(bill);
console.log(formatBill(bill));

// Error tests
try {
    calculateBill({ amount: -100, tipPercent: 10, people: 2 });
} catch (e) {
    console.log("Caught:", e.message);
}

try {
    calculateBill({ amount: 1500, tipPercent: 10, people: 0 });
} catch (e) {
    console.log("Caught:", e.message);
}

try {
    calculateBill({ amount: 1000, tipPercent: "ten", people: 2 });
} catch (e) {
    console.log("Caught:", e.message);
}

```

6. In-Class Lab: Finish Fundamentals (lab.js)

Consolidated all lab checklist requirements into one script. Runs with zero uncaught errors.

File: Week 5/lab.js

```

// In-Class Lab: Finish the Fundamentals File (lab.js)

// 1. Helpers
const getInitials = (name) =>
    name.trim().split(/\s+/).map((w) => w[0].toUpperCase()).join("");

const isPalindrome = (str) => {
    const s = str.toLowerCase().replace(/[^a-z0-9]/g, "");
    return s.length > 0 && s === s.split("").reverse().join("");
};

const countWords = (str) =>
    str.trim() === "" ? 0 : str.trim().split(/\s+/).length;

console.log("Initials:", getInitials("Ayesha Khan"));
console.log("Palindrome:", isPalindrome("Race car"));
console.log("Words:", countWords("the quick brown fox"));

```

```
// 2. Above-average students (no loops)
const students = [
  { name: "Ali", score: 82 },
  { name: "Sara", score: 95 },
  { name: "Ahmed", score: 68 },
  { name: "Zainab", score: 77 },
  { name: "Bilal", score: 91 },
  { name: "Hamza", score: 45 }
];
const avg = students.reduce((sum, s) => sum + s.score, 0) / students.length;
const aboveAvg = students.filter((s) => s.score > avg).map((s) => s.name);
console.log("Average:", avg.toFixed(2));
console.log("Above Average:", aboveAvg);

// 3. groupBy using reduce
const products = [
  { title: "Keyboard", category: "peripherals" },
  { title: "Mouse", category: "peripherals" },
  { title: "USB Cable", category: "cables" }
];
const groupBy = (arr, key) =>
  arr.reduce((acc, item) => {
    (acc[item[key]] = acc[item[key]] || []).push(item);
    return acc;
  }, {});
console.log("Grouped:", groupBy(products, "category"));

// 4. calculateBill with validation + passing test for each error case
function calculateBill({ amount, tipPercent = 15, people = 1 } = {}) {
  if (typeof amount !== "number" || amount <= 0) {
    throw new Error("Invalid amount");
  }
  if (typeof tipPercent !== "number" || tipPercent < 0) {
    throw new Error("Invalid tip percentage");
  }
  if (!Number.isInteger(people) || people < 1) {
    throw new Error("Invalid people count");
  }

  const tip = amount * (tipPercent / 100);
  const total = amount + tip;
  return {
    amount,
    tipPercent,
    total: Number(total.toFixed(2)),
    perPerson: Number((total / people).toFixed(2))
  };
}

console.log("Bill:", calculateBill({ amount: 2400, tipPercent: 10, people: 3 }));

try {
  calculateBill({ amount: -50, people: 2 });
} catch (e) { }
```

```

    console.log("Caught:", e.message);
  }

  try {
    calculateBill({ amount: 1000, people: 0 });
  } catch (e) {
    console.log("Caught:", e.message);
  }

  try {
    calculateBill({ amount: 1000, tipPercent: "ten", people: 2 });
  } catch (e) {
    console.log("Caught:", e.message);
  }
}

```

7. Homework Task 1: Gradebook Script (gradebook.js)

Computed class average, highest/lowest scorer, pass rate, and sorted ranking using only array methods.

File: Week 5/gradebook.js

```

// Homework Task 1: Gradebook Script
const students = [
  { name: "Muhammad Tayyab", score: 94 },
  { name: "Ayesha Khan", score: 88 },
  { name: "Bilal Ahmed", score: 76 },
  { name: "Zainab Fatima", score: 91 },
  { name: "Hamza Tariq", score: 48 },
  { name: "Sara Noor", score: 83 },
  { name: "Usman Ali", score: 62 },
  { name: "Dua Malik", score: 42 }
];

const total = students.reduce((sum, s) => sum + s.score, 0);
const avg = total / students.length;
const highest = students.reduce((max, s) => (s.score > max.score ? s : max));
const lowest = students.reduce((min, s) => (s.score < min.score ? s : min));
const passRate = (students.filter((s) => s.score >= 50).length / students.length) * 100;
const sorted = [...students].sort((a, b) => b.score - a.score);

console.log("Class Average:", avg);
console.log("Highest Scorer:", highest.name, `${highest.score}`);
console.log("Lowest Scorer:", lowest.name, `${lowest.score}`);
console.log("Pass Rate:", `${passRate}%`);
console.log("Sorted:", sorted.map((s) => `${s.name}: ${s.score}`));

```

8. Homework Task 2: Predict Before You Run (predict.txt)

Predictions vs actual output for type coercion, strict equality, and array sorting.

File: Week 5/predict.txt

Task 2 Predictions and Explanations

```
1. 0 == false
Prediction: true
Output: true
Reason: Loose equality coerces false to 0, so 0 == 0.

2. 0 === false
Prediction: false
Output: false
Reason: Strict equality checks type without coercion (number !== boolean).

3. [] == false
Prediction: true
Output: true
Reason: [] converts to "" which coerces to 0, so 0 == 0.

4. [] == ![]
Prediction: true
Output: true
Reason: ![] is false because arrays are truthy. Then [] == false becomes true.

5. null == undefined
Prediction: true
Output: true
Reason: In JavaScript null and undefined are loosely equal by specification.

6. null === undefined
Prediction: false
Output: false
Reason: Different types (null vs undefined).

7. typeof NaN
Prediction: number
Output: number
Reason: In JS, NaN is still classified as a numeric type.

8. 0 || 42 vs 0 ?? 42
Prediction: 42 vs 0
Output: 42 vs 0
Reason: || treats 0 as falsy, but ?? only checks for null or undefined.

9. [10, 2, 5, 1].sort()
Prediction: [1, 10, 2, 5]
Output: [1, 10, 2, 5]
Reason: Default sort converts elements to strings and sorts alphabetically ("10" comes before "2").
```